

TRABAJO PRACTICO N.º 2

Laboratorio: Concurrencia e IA como motor primario

Materia	Base de Datos II
Alumno	Nicolas Moron
Proyecto integrador	Food Store - sistema de gestion de pedidos
Motor	PostgreSQL 18.6
Fecha de verificacion	10/09/2026

Alcance de la entrega

Este documento reúne la resolución de las Partes 0, 1, 2 y 3 de la guía. Las pruebas se ejecutaron sobre la base aislada **foodstore_tp2**, con dos conexiones psql independientes. Las salidas consignadas provienen de PostgreSQL 18.6.

Repositorio

Código fuente, scripts y evidencias: https://github.com/Nixus32xD/DB_Nicolas_Moron

Resumen de resultados

Parte	Resultado
0 - Seguridad	Copia de trabajo, transacciones de prueba y respaldo previo al DDL.
1 - Integridad	Dos CHECK y un UNIQUE aplicados y comprobados con casos validos e invalidos.
2 - Concurrencia	Lectura no repetible, lectura fantasma y espera por bloqueo reproducidas en dos sesiones.
3 - Lectura critica	Analisis del UPDATE sin WHERE y del riesgo de NOT IN ante NULL.

0. Puesta a punto y protocolo de seguridad

El trabajo se realizó sobre la base local **foodstore_tp2**, creada exclusivamente para el laboratorio. No se usó una base de producción ni datos de terceros.

Copia, transaccion y respaldo

```
createdb foodstore_tp2
psql -d foodstore_tp2 -f schema.sql
psql -d foodstore_tp2 -f sql/00_datos_iniciales.sql
pg_dump -Fc foodstore_tp2 > backups/foodstore_tp2_pre_restricciones_20260910.dump
```

Antes de cada operación se verificó la conexión activa:

```
SELECT current_database(), current_user;
-- foodstore_tp2 | postgres
```

Las restricciones se probaron primero en una transacción con **BEGIN ... ROLLBACK**. Luego se aplicaron en la copia de trabajo. El respaldo custom generado por `pg_dump` fue validado con `pg_restore -l`.

Uso de IA

Por elección del alumno, Codex cumplió el rol de agente de código. Se siguió el flujo: planificar, revisar el diff, ejecutar sobre una copia y confirmar los resultados con PostgreSQL. El archivo **AGENTS.md** y **.kiro/steering/food-store.md** dejan versionadas las convenciones del proyecto.

Datos mínimos reproducibles

Se agregó **sql/00_datos_iniciales.sql** porque las pruebas usan los IDs 1, 2 y 3. Carga 2 categorías, 2 clientes, 3 productos, 2 pedidos y 1 detalle inicial; así los experimentos se pueden repetir desde una base nueva.

1. Integridad versionada

Reglas de negocio implementadas sobre **pedido_detalle**:

Regla	Implementacion	Resultado
Cantidad	CHECK (cantidad > 0)	Una línea de pedido requiere al menos una unidad.
Precio	CHECK (precio_unitario >= 0)	El precio historico no puede ser negativo.
Producto repetido	UNIQUE (pedido_id, producto_id)	Un producto no puede figurar dos veces en el mismo pedido.

Aplicacion transaccional

```
BEGIN;  
ALTER TABLE  
ALTER TABLE  
ALTER TABLE  
ROLLBACK;
```

Aplicacion final sobre la copia:

```
ALTER TABLE  
ALTER TABLE  
ALTER TABLE
```

Casos ejecutados

Caso	Salida real
Valido	INSERT 0 1. Se insertó pedido_id=2, producto_id=1, cantidad=1 y precio_unitario=1050.00; luego ROLLBACK.
Cantidad 0	ERROR: viola chk_pedido_detalle_cantidad_positiva.
Precio -50	ERROR: viola chk_pedido_detalle_precio_no_negativo.
Duplicado	ERROR: viola uq_pedido_detalle_pedido_producto; clave (1, 1) ya existente.

Conclusión

Los tres mecanismos declarativos quedaron presentes en el catálogo de PostgreSQL y rechazaron los registros inválidos con el error esperado.

2. Laboratorio de concurrencia

Se utilizaron dos conexiones psql concurrentes, denominadas Sesión A y Sesión B. Se mantuvo la Sesión A abierta mediante una pausa controlada mientras B confirmaba su operación.

2.1 Lectura no repetible

```
Sesion A: BEGIN; SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT precio FROM producto WHERE id = 1;
Sesion B: BEGIN; UPDATE producto SET precio = precio + 100 WHERE id = 1; COMMIT;
Sesion A: SELECT precio FROM producto WHERE id = 1; COMMIT;
```

Nivel	Primera lectura	Segunda lectura	Resultado
READ COMMITTED	1050.00	1150.00	La misma transaccion observó el COMMIT de B.
REPEATABLE READ	1050.00	1050.00	El snapshot de A se mantuvo estable.

Conclusión: la explicación se confirmó. En READ COMMITTED PostgreSQL toma un snapshot por sentencia; REPEATABLE READ conserva el snapshot de la transacción.

2.2 Lectura fantasma

```
Sesion A: BEGIN; SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT COUNT(*) FROM producto WHERE categoria_id = 1 AND activo = TRUE;
Sesion B: BEGIN; INSERT INTO producto (...) VALUES (...); COMMIT;
Sesion A: SELECT COUNT(*) FROM producto WHERE categoria_id = 1 AND activo = TRUE; COMMIT;
```

Nivel	COUNT inicial	COUNT posterior	Resultado
READ COMMITTED	2	3	A observó la fila insertada y confirmada por B.
REPEATABLE READ	3	3	A no observó la fila insertada después de su snapshot.

2.3 Espera por bloqueo

```
Sesion A: BEGIN; SELECT id, stock FROM producto WHERE id = 1 FOR UPDATE;
-- mantiene el lock durante 3 segundos; COMMIT;
Sesion B: BEGIN; SELECT id, stock FROM producto WHERE id = 1 FOR UPDATE; COMMIT;
```

Evento	Salida real
Sesión A	Obtuvo el lock de producto id=1 con stock=20.
Sesión B - intento	08:53:23.855485-03
Sesión B - lock obtenido	08:53:25.887246-03
Espera observada	Aproximadamente 2.032 segundos, hasta el COMMIT de A.

Conclusión: la explicación se confirmó. SELECT ... FOR UPDATE adquiere un bloqueo incompatible sobre la fila y la segunda sesión espera hasta que la primera libera el lock con COMMIT o ROLLBACK.

Síntesis de la Parte 2

Los tres escenarios obligatorios se reprodujeron en el motor real. Las salidas completas están en **evidencias/02_concurrencia_postgresql_18.txt** y en **informe_concurrencia.md**. El interbloqueo es opcional y no forma parte de esta entrega.

3. Lectura crítica de scripts SQL

Script 1 - UPDATE sin WHERE

```
UPDATE funcion
SET activa = FALSE;
```

Efecto real: desactivaría **todas** las filas de la tabla funcion, porque no restringe el conjunto a las funciones vinculadas con películas retiradas. La corrección debe usar la condición real del esquema y verificarse primero con un SELECT.

```
SELECT * FROM funcion
WHERE [condicion_real_que_identifica_funciones_retiradas];

UPDATE funcion
SET activa = FALSE
WHERE [condicion_real_que_identifica_funciones_retiradas];
```

Script 2 - NOT IN frente a NULL

```
DELETE FROM categoria
WHERE id NOT IN (SELECT categoria_id FROM producto);
```

Riesgo: si la subconsulta devuelve al menos un NULL, la lógica ternaria puede hacer que el filtro sea UNKNOWN y no seleccione las filas esperadas. La forma robusta es NOT EXISTS. En Food Store se prefiere la baja lógica para conservar el historial.

```
UPDATE categoria c
SET activo = FALSE
WHERE c.activo = TRUE
AND NOT EXISTS (
  SELECT 1 FROM producto p
  WHERE p.categoria_id = c.id
);
```

Conclusión

La IA aporta propuestas, pero la decisión queda a cargo de quien implementa. La verificación contra el motor, el uso de una copia, las transacciones y el respaldo son controles necesarios antes de confirmar un cambio.

4. Declaración de Uso de IA y trazabilidad

Parte	Uso registrado	Verificación
Parte 1	Codex propuso restricciones y casos de prueba siguiendo el flujo plan, diff y ejecución.	Casos válidos e inválidos en PostgreSQL 18.6.
Parte 2	Codex explicó MVCC, snapshots y locks para los escenarios reproducidos.	Dos sesiones psql; resultados conservados en evidencias.
Parte 3	Codex apoyó el análisis de alcance y riesgo de los scripts.	Revisión manual; no se ejecutaron scripts peligrosos.

Historial relevante

```
6a2e13c chore: inicializar repositorio y esquema Food Store
83eb2b9 feat: agregar restricciones de integridad de Food Store
5348e63 test: documentar escenarios de concurrencia en PostgreSQL
39f9e15 test: verificar restricciones y concurrencia en PostgreSQL 18
094ef5a docs: agregar steering y flujo de Codex
fe7d0f8 docs: agregar resoluciones PDF de TP1 y TP2
```

Archivos de respaldo de la entrega

El repositorio contiene schema.sql, sql/00_datos_iniciales.sql, los scripts de restricciones y concurrencia, protocolo_seguridad.md, las tres DUIA, informe_concurrencia.md, ejercicio_lectura_critica.md y evidencias de PostgreSQL. El enlace público está incluido en la primera página.

Fin de la entrega.